Design Principles

1. Single Responsibility Principle (SRP)

Modules	PIC (Person(s) in charge)	Description	Improvement Direction
PlaceOrderService	Nguyễn Huy Diễn	The service coordinates draft creation, invoice generation, order finalization, order retrieval, and cancellation. Changes to any of these workflows may require modification of the same class.	Consider separating responsibilities into dedicated application services such as OrderDraftService, OrderFinalizationService, and OrderQueryService if the workflow becomes more complex.
ProductManagementService	Lê Hoàng Kiên	The class is handling too many responsibilities: 1. Coordinating CUD business logic (Write). 2. Handling data retrieval logic (Read). 3. Directly containing the low-level utility function to handle Java Reflection.	Segregate the interface into ProductCommandService (for CUD) and ProductQueryService (for data retrieval). At the same time separate the utility function into a distinct utility class.
PayOrderService	Nguyễn Tiến Đạt	confirmPayment() handles two distinct concerns: verifying the VietQR payment status and triggering order finalization, which belongs to a different domain. Changes to the order lifecycle also force this payment service to change.	Move the orderFinalization call to a dedicated CheckoutOrchestrator that coordinates PayOrderService and OrderFinalization independently.
PayByCreditCardService	Phạm Đức Phước	The old service had one method mixing card validation, a hardcoded "starts with-4" gateway	(Already refactored) Gateway logic moved behind the IPaymentProvider Facade; persistence + email delegated to the existing

		simulation, transaction persistence, and confirmation-email sending — four reasons to change, low cohesion.	PlaceOrderService.finalizeOrder(); service reduced to a thin create/capture orchestrator. PayPal concerns split into PayPalAuthClient, PayPalOrdersClient, PayPalAmountConverter

2. Open/Closed Principle (OCP)

Modules	PIC (Person(s) in charge)	Description	Improvement Direction
PlaceOrderService	Nguyễn Huy Diễn	Order completion requirements are hardcoded in the finalization logic. Adding new mandatory order components would require modification of existing code.	Introduce extensible order validation strategies or completion rules.
DeliveryService	Nguyễn Huy Diễn	Delivery fee calculation depends on a concrete fee-calculation implementation. Changes to fee-calculation policies may require modification of existing classes.	Introduce a DeliveryFeeStrategy abstraction and move calculation algorithms into separate implementations.
ProductFactory	Lê Hoàng Kiên	To create or update specific objects (Book, CD, DVD, Newspaper), the current Factory is highly likely using a switch-case structure based on productType. If AIMS introduces a new product type, developers must open the ProductFactory file and add a new case. This directly violates OCP.	Replace the switch-case structure with the Registry or Strategy pattern. Create an IProductBuilder interface with a build() method. Classes like BookBuilder and CDBuilder will implement this interface. In ProductFactory, use a Map<ProductType, IProductBuilder> to flexibly register builders without modifying existing

			code when a new type is added.
PayByCreditCardService	Phạm Đức Phước	Payment success was decided by a hardcoded rule baked into the service, so adding a real gateway or a second provider required modifying working code (cascading changes)	(Already refactored) Introduced IPaymentProvider; each gateway is a separate implementation, added by extension with no edits to the service or controller.
PaymentTransaction.Method (current)	Phạm Đức Phước	Minor: the payment-method enum {VIETQR, PAYPAL} must be edited to add a provider. Localized only — the provider call path is OCP clean via IPaymentProvider.	Acceptable for a known, closed set; if methods become open-ended, derive the method code from the IPaymentProvider implementation instead of the enum.

3. Liskov Substitution Principle (LSP)

Modules	PIC (Person(s) in charge)	Description	Improvement Direction

4. Interface Segregation Principle (ISP)

Modules	PIC (Person(s) in charge)	Description	Improvement Direction

5. Dependency Inversion Principle (DIP)

Modules	PIC (Person(s) in charge)	Description	Improvement Direction
DeliveryService	Nguyễn Huy Diễn	The service depends directly on the concrete DeliveryFeeCalculator implementation. High-level delivery processing logic is	Depend on a DeliveryFeeStrategy abstraction instead of a concrete calculator implementation.

		coupled to a specific fee-calculation mechanism.	
OrderController	Nguyễn Huy Diễn	The controller depends directly on the concrete service classes DeliveryService and PlaceOrderService. This issue is of low-severity concern since Spring service layer dependencies are common.	May introduce use-case-specific interfaces and inject abstractions.
ProductManagementService	Lê Hoàng Kiên	ProductManagementService (High-level module) directly depends on and injects ProductFactory (Concrete class / Low-level module). When unit testing the Service class, mocking a concrete Factory becomes difficult, creating tight coupling between the two modules.	1. Create an interface named IProductFactory. 2. Make the current ProductFactory class implements IProductFactory. 3. In ProductManagementService, change the dependency declaration type to: private final IProductFactory productFactory;.
PayByCreditCardService (legacy)	Phạm Đức Phước	The old high-level use case depended directly on concrete low-level types (two concrete repositories + a concrete notification service), making the gateway un-swappable and the class hard to test.	(Already refactored) Now depends on the IPaymentProvider abstraction and PlaceOrderService, injected via constructor (DIP satisfied).
IPaymentProvider (current — residual)	Phạm Đức Phước	Ownership inversion: the abstraction the core depends on lives inside the low-level subsystems.paypal package, so the core imports its contract from the provider's package.	Move IPaymentProvider + PaymentInitiation + PaymentCapture into a core owned package (e.g. ...payment.port); the PayPal subsystem then depends inward on the core's contract.
PayByCreditCardService (current — low severity)	Phạm Đức Phước	Depends on the concrete PlaceOrderService and OrderDraftContext. Low severity (stable Spring beans), same category as the OrderController note.	Optionally depend on narrow ports (OrderFinalizer, OrderDraftSource) for finer testability.